

Concurrent Task Scheduler in Rust - Design Report

Overview

This project implements a concurrent task scheduling simulator in Rust. The goal of the system is to model how a runtime or operating system scheduler might manage CPU-bound and IO-bound tasks using multiple worker threads, shared queues, synchronization primitives, and scheduling policies.

The scheduler uses a bounded worker pool, queue-based task management, and concurrent execution to process workloads under different conditions. The project was designed to emphasize concurrency concepts such as producer-consumer behavior, synchronization, scheduling tradeoffs, shared state protection, and clean shutdown. Two experiments were conducted: a balanced workload containing a roughly equal mix of CPU and IO tasks, and a stressed workload dominated by long-running CPU tasks.

The results demonstrate how scheduling policies influence throughput, fairness, wait times, and worker utilization.

System Architecture

The system is divided into several concurrent components, each with a clearly defined responsibility. The main thread coordinates the overall execution of the program. It creates shared resources, launches worker threads, starts the task generator, waits for completion, and handles clean shutdown.

The generator thread acts as the producer in the producer-consumer model. It creates simulated tasks and inserts them into the scheduler queues. Each task contains: a task ID, a task type (CPU or IO), a simulated duration, and a creation timestamp. CPU tasks are generally assigned longer durations than IO tasks to simulate realistic workloads.

The system uses a bounded worker pool of six worker threads. Workers repeatedly pull tasks from the shared queues and simulate execution using thread sleep. Workers update shared metrics after task completion and continue accepting work until shutdown occurs.

The scheduler uses two shared FIFO queues: a CPU queue, and an IO queue. Separating task classes allows the scheduling policy to treat CPU and IO workloads differently. A shared metrics structure records: total completed tasks, CPU tasks completed, IO tasks completed, average wait time, average turnaround time, makespan, worker utilization, and maximum wait time.

Shared Data and Synchronization

Several structures are shared between multiple threads. The CPU and IO queues are shared between the generator thread and worker threads. The queues are protected using: `"Arc<Mutex<VecDeque<Task>>>"`. ``Arc`` allows multiple threads to own the same queue safely, while ``Mutex`` ensures that only one thread modifies the queue at a time. This prevents race conditions when workers remove tasks concurrently.

The metrics structure is also shared between all workers. It is protected using: “Arc<Mutex<Metrics>>”. This ensures updates to timing statistics and counters remain consistent.

Shutdown state is coordinated using: “Arc<AtomicBool>”. Workers periodically check this flag to determine when they should terminate.

Channels and Shared State

The original design considered using Rust channels between the generator and dispatcher because channels naturally support producer-consumer communication. However, the final implementation uses shared queues directly instead of channels. This simplified the architecture and reduced unnecessary forwarding complexity.

Shared state was ultimately preferred because all workers require visibility into the same queues, dynamic task pulling is simpler with shared queues, and the system remains easier to explain during demonstration. Although channels were not used in the final implementation, the project still demonstrates concurrent producer-consumer behavior through shared synchronized queues.

Scheduling Policy

The scheduler implements a weighted FIFO scheduling policy with reserved CPU workers. Tasks inside each queue are processed in first-in-first-out order. FIFO scheduling was chosen because: it is simple to implement, behavior is predictable, and fairness is reasonable under balanced workloads.

Workers 0 and 1 are reserved for CPU tasks only. The remaining workers are flexible workers that refer to IO tasks first and execute CPU tasks if no IO tasks are available. This creates a weighted scheduling system that prioritizes responsiveness for IO workloads while still maintaining progress on CPU-heavy workloads.

Scheduling Tradeoffs

The scheduling policy improved several aspects of system behavior. IO tasks experienced lower wait times because flexible workers prioritized IO queues first. Separating CPU and IO queues prevented long-running CPU tasks from completely blocking short IO tasks. During balanced workloads, worker utilization remained consistently high and both task classes made steady progress. Reserved CPU workers ensured CPU tasks continued executing even during IO-heavy bursts. The policy also introduced additional complexity. Reserved CPU workers may remain idle during workloads dominated by IO tasks. Workers must examine multiple queues and follow different scheduling rules. FIFO ordering can still allow short tasks to become trapped behind very long tasks. The scheduler reduces starvation risk but does not eliminate it entirely.

Concurrency Issues During Development

One of the main issues encountered during development involved shutdown handling. Initially, workers executed in an infinite loop waiting for tasks forever. Once all tasks completed, the program would hang because workers had no mechanism for detecting termination. Another

issue involved queue synchronization. Without proper locking, multiple workers could attempt to modify queues simultaneously, potentially causing race conditions.

The shutdown issue was fixed using an atomic shutdown flag: “Arc<AtomicBool>”. After task generation completed and all queues became empty, the main thread set the shutdown flag. Workers periodically checked the flag and exited their loops cleanly. Race conditions were prevented by protecting queues and metrics using mutexes. Additionally, small sleep intervals were added when queues were empty to prevent excessive busy waiting and unnecessary CPU consumption.

Experiment A — Balanced Workload

- 500 total tasks
- 50% CPU tasks
- 50% IO tasks
- moderate task durations
- 6 worker threads

The balanced workload demonstrated stable throughput and good worker utilization. IO tasks maintained relatively low wait times because flexible workers prioritized the IO queue. Average turnaround time remained moderate, and both queues drained steadily throughout execution. The scheduling policy handled balanced workloads effectively without significant starvation.

Experiment B — Stressed Workload

- 750 total tasks
- 85% CPU tasks
- long CPU durations
- 6 worker threads

The stressed workload significantly increased average wait time and makespan. The CPU queue grew substantially larger than the IO queue, causing longer delays for CPU-bound tasks. However, the reserved CPU workers prevented complete queue collapse and ensured steady CPU progress. IO tasks were still completed relatively quickly because flexible workers prioritized IO work whenever available. The experiment demonstrated how workload imbalance can impact fairness and throughput even with multiple scheduling safeguards.

Remaining Fairness and Starvation Risks

Although the scheduler improves fairness compared to a single shared FIFO queue, starvation is still possible under certain conditions. Examples include: extremely CPU-heavy workloads, continuous arrival of one task type, and very long-running CPU tasks. FIFO ordering can also create convoy effects where short tasks wait behind very long tasks. Future improvements could include: aging, priority boosting, shortest-job-first scheduling, weighted round robin, and work stealing between workers.

Conclusion

This project demonstrates how concurrency primitives, scheduling policies, and synchronization strategies can be combined to build a functional task scheduling system in Rust. The final design successfully implemented concurrent execution, queue-based scheduling, a bounded worker pool, workload separation, metrics collection, clean shutdown, and experimental analysis. The project also highlighted important concurrency tradeoffs involving fairness, synchronization overhead, worker utilization, and scheduling complexity. Most importantly, the system maintained a clear and explainable task lifecycle from generation through execution and shutdown, making the architecture both functional and understandable during analysis and demonstration.